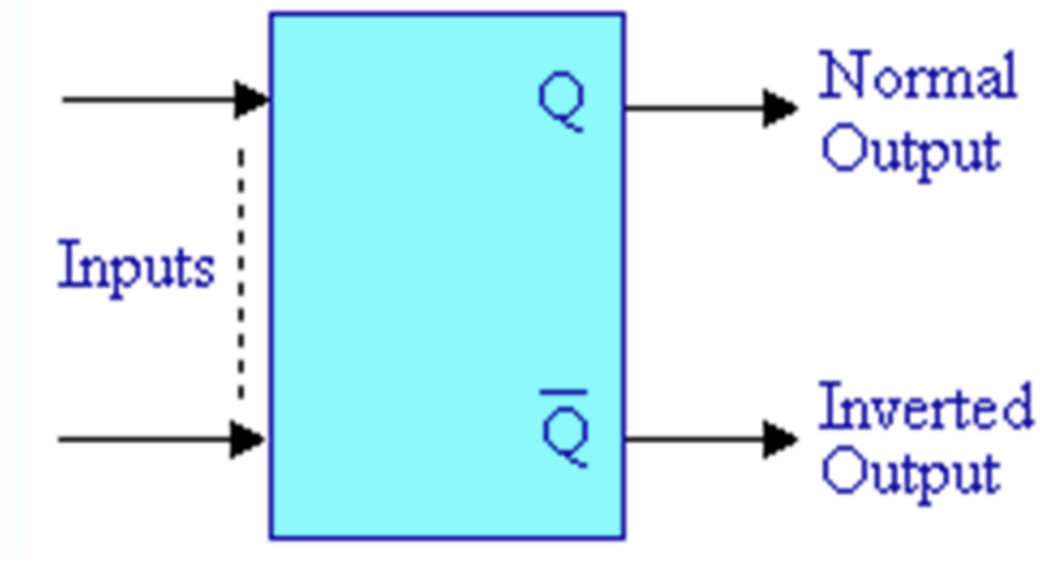
Intro to Bistables

Previously, we have focused on Combinational Logic Circuits. This topic covers Sequential Logic Circuits, where time and order of the inputs does matter for the outputs, not just the combination of them.

Bistables are circuits that are either latches or flip flops, with 2 stable states. These circuits are usually able to store 1 bit of memory.

Latches and Flip-Flops

- Both latches and flip flops store memory, however flip flops only store data based on the clock signal, while latches do not.
- However, both have 2 inputs, one for data/to set the latch (called set, or S) and another to clear the latch (called C, or R for reset)
- thus, these latches are called SC latches (or SR latches)
- Active High/Active Low Inputs
  - Active High refers to the inputs needing to be 1, or HIGH, for the intended action.
  - This is the more intuitive, like how you have to switch on a light to high.
  - Active Low refers to the inputs needing to be 0, or LOW, for the intended action.
  - So applying a 1 does not do anything, but 0 will cause the intended action.
- Flip flops build on SC latches, with other logic gates to include the clock signal.
- These latches/flip flops also have 2 outputs, Q and $\bar{Q}$, which should always be at different logic levels.



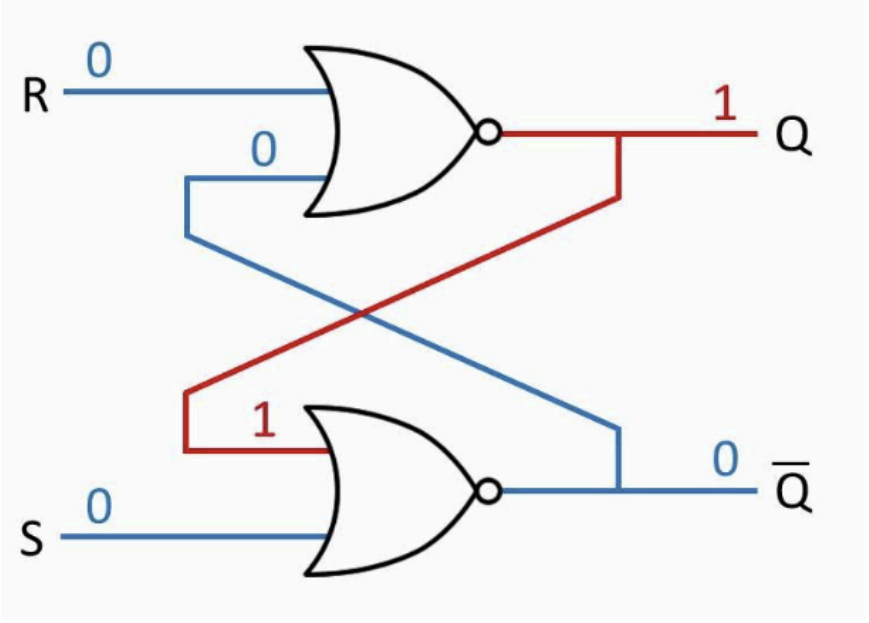
We usually describe setting and clearing a latch with these outputs, where setting a latch is forcing Q to be high, and resetting a latch is forcing Q to be low.

SR/SC Latch

A Set-Reset, or Set Clear Latch, is the most basic type of latch, and can be made with NOR or NAND gates.

NOR Latch

A NOR latch is implemented like so:



The NOR latch has active high inputs, so when S=1 and C = 0, Q will be forced to be 1.

Conversely, $\bar{Q}$ will be low when S=1.

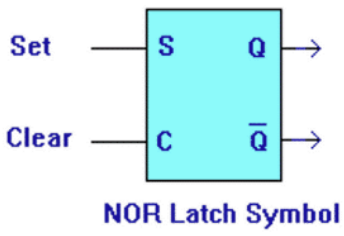
Likewise, when C = 1 and S = 0, Q will be cleared; forced to 0, and $\bar{Q}$ = 1.

When S = 0 and C = 0, Q and $\bar{Q}$ retain the same state (assuming the previous state was not S = 1 and C = 1), also called the rest state.

However, when S = 1 and C = 1, both Q and $\bar{Q}$ = 0, which breaks the convention of Q being opposite to $\bar{Q}$, thus this state is invalid.

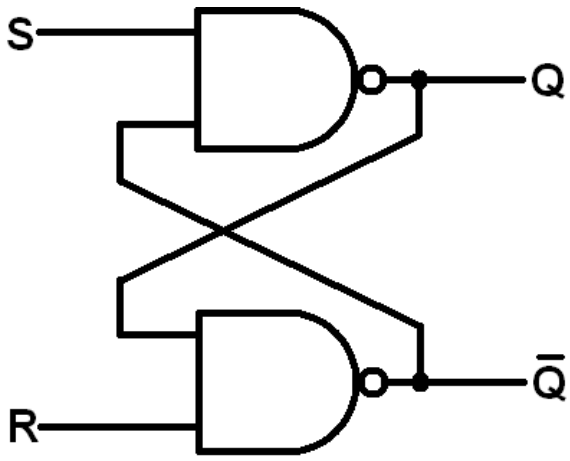
The following can be summarised in this truth table:

Set	Clear	Q	$\bar{Q}$	Remarks
0	0	Q_0	Q_0	no change
1	0	1	0	Set
0	1	0	1	Clear
1	1	0	0	Invalid



NAND Latch

A NAND Latch is implemented like so:



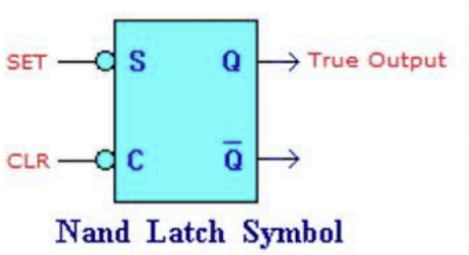
A NAND Latch has active low inputs, so when S = 0 and R = 1, S is active and R is non active, thus Q is set, causing Q = 1 and $\bar{Q}$ = 0.

Likewise, when S = 1 and R = 0, R is active, thus Q = 0 and $\bar{Q}$ = 1.

The rest state of the NAND latch is when S = R = 1 (both inactive), causing Q and $\bar{Q}$ retain the same state,

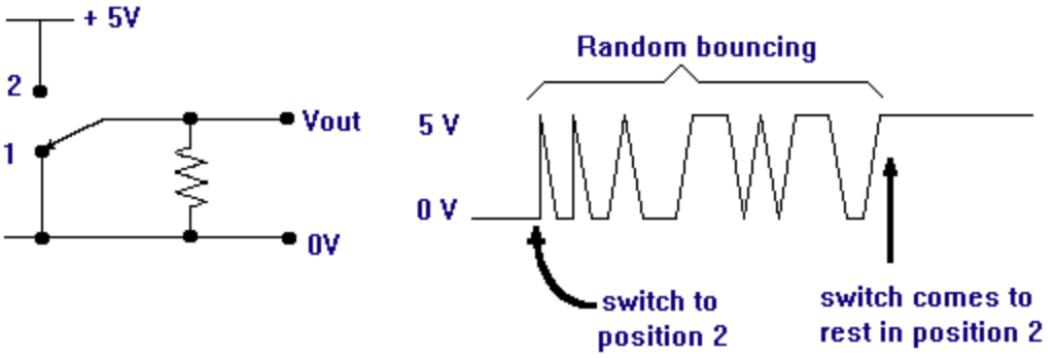
while the invalid state is when S = R = 0 (active low), and causes Q = $\bar{Q}$ = 1.

S	C	Q	$\bar{Q}$	Remarks
1	1	Q	$\bar{Q}$	No Change
1	0	0	1	Clear
0	1	1	0	Set
0	0	1	1	Indeterminate

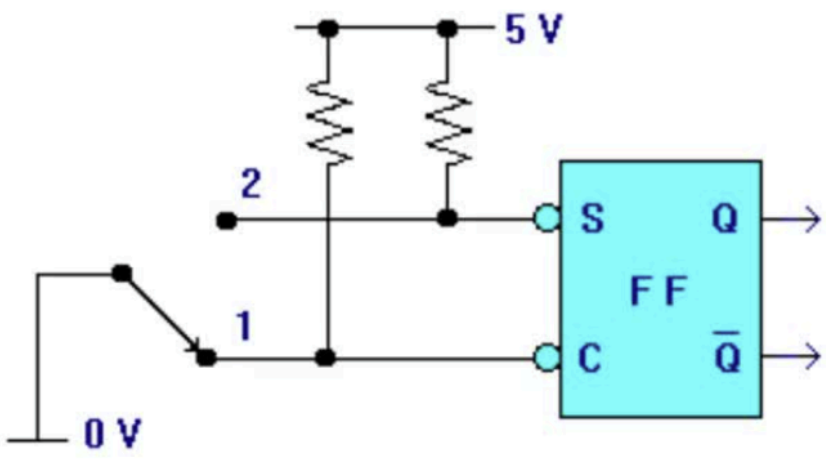


Applications

One use for latches is switch debouncing, where the random signal transitions between high and low in switches are removed.



By connecting an SR latch like so:



at the first instance of the switch turning on, the S in the latch is activated and the output Q stores this, so any other switch bouncing does not cause noise.